

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

fiktus_taskone_path = kagglehub.dataset_download('fiktus/taskone')

print('Data source import complete.')
```

Школа глубокого обучения ФПМИ МФТИ

Домашнее задание. Детекция объектов

В этом домашнем задании мы продолжим работу над детектором из семинара, поэтому при необходимости можете заимствовать оттуда любой код.

Домашнее задание можно разделить на следующие части:

- Переделываем модель [4]
 - Backbone [1],
 - Neck [2],
 - Head [1]
- Label assignment [3]:
 - TAL [3]
- Лоссы [1]:
 - CloU loss [1]
- Кто больше? [5]
 - 0.05 mAP [1]
 - 0.1 mAP [2]
 - 0.2 mAP [5]

Максимальный балл: 10 баллов. (+3 балла бонус).

```
import torch
import numpy as np
import pandas as pd
import albumentations as A

from PIL import Image
from torchvision import transforms
from torch.utils.data import Dataset
from albumentations.pytorch.transforms import ToTensorV2
import math
from functools import partial
from collections import Counter, defaultdict

import io
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import timm
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset
from torch.utils.data import DataLoader

from PIL import Image
from tqdm.auto import tqdm
from torchvision import transforms
from torchvision.ops import nms, box_iou
from torchvision.models.detection.anchor_utils import AnchorGenerator

from torchmetrics.detection import MeanAveragePrecision
```

Загрузка данных

Мы продолжаем работу с датасетом из семинара - Halo infinite ([ссылка](#)). Загрузка данных и создание датасета полностью скопированы из семинара.

Сначала загружаем данные

```
splits = {'train': 'data/train-00000-of-00001-0d6632d599c29801.parquet',
          'validation': 'data/validation-00000-of-00001-c6b77a557eedd52.parquet',
          'test': 'data/test-00000-of-00001-866d29d8989ea915.parquet'}
df_train = pd.read_parquet("hf://datasets/Francesco/halo-infinite-angel-videogame/" + splits["train"])
df_test = pd.read_parquet("hf://datasets/Francesco/halo-infinite-angel-videogame/" + splits["test"])
```

df_train

image_id		image	width	height	objects	
0	311	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [573, 574], 'area': [1748, 15756], 'bbo...	
1	67	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [137, 138], 'area': [14136, 88392], 'bb...	
2	161	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [301, 302], 'area': [3016, 33768], 'bbo...	
3	210	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [379, 380], 'area': [832, 9248], 'bbox'...	
4	142	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [272, 273], 'area': [1520, 16704], 'bbo...	
...	...		...	...		...
457	17	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [33], 'area': [6160], 'bbox': [[312.0, ...	
458	193	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [356, 357, 358, 359], 'area': [3472, 78...	
459	428	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [783, 784], 'area': [6160, 65520], 'bbo...	
460	211	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [381, 382], 'area': [2700, 43152], 'bbo...	
461	410	{'bytes': b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x...	640	640	{'id': [755], 'area': [9984], 'bbox': [[320.0,...	
462 rows × 5 columns						

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.

<pre>dataframe = df_train df_objects = pd.json_normalize(dataframe['objects'])[["bbox", "category"]] df_images = pd.json_normalize(dataframe['image'])[["bytes"]] df_ofs = dataframe[["image_id"]].join(df_objects).join(df_images) df_ofs</pre>						
image_id		bbox	category	bytes		
0	311	[[248.0, 228.0, 38.0, 46.0], [232.0, 228.0, 78...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
1	67	[[520.0, 278.0, 114.0, 124.0], [386.0, 270.0, ...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
2	161	[[340.0, 48.0, 52.0, 58.0], [298.0, 48.0, 126...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
3	210	[[310.0, 272.0, 26.0, 32.0], [286.0, 272.0, 68...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
4	142	[[304.0, 300.0, 40.0, 38.0], [276.0, 300.0, 96...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
...	...	...	...			...
457	17	[[312.0, 318.0, 44.0, 140.0]]	[1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
458	193	[[396.0, 224.0, 62.0, 56.0], [578.0, 304.0, 28...	[4, 2, 1, 3]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
459	428	[[266.0, 156.0, 70.0, 88.0], [206.0, 176.0, 18...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
460	211	[[372.0, 302.0, 50.0, 54.0], [260.0, 304.0, 18...	[2, 1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
461	410	[[320.0, 352.0, 96.0, 104.0]]	[1]	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...		
462 rows × 4 columns						

<pre>import io row = df_ofs.iloc[1] image = Image.open(io.BytesIO(row["bytes"])) image = np.array(image) labels = [row["category"]] if isinstance(row["category"], int) else row['category'] # Вычитаем единицу чтобы классы начинались с нуля labels = [label - 1 for label in labels] labels</pre>	
[np.int64(1), np.int64(0)]	

image.shape
(640, 640, 3)

Создаем датасет для предобработки данных

<pre>class HaloDataset(Dataset): def __init__(self, dataframe, transform=None): df_objects = pd.json_normalize(dataframe['objects'])[["bbox", "category"]] df_images = pd.json_normalize(dataframe['image'])[["bytes"]] self.data = dataframe[["image_id"]].join(df_objects).join(df_images) self.transform = transform def __len__(self): return len(self.data) def __getitem__(self, idx): row = self.data.iloc[idx] image = Image.open(io.BytesIO(row["bytes"])) image = np.array(image) target = {"image_id": row["image_id"]} # классы: в датасете идут 1..4, приводим к 0..3 (без фона) labels = [row["category"]] if isinstance(row["category"], int) else row['category'] labels = [label - 1 for label in labels] # bbox в исходных данных: [x_min, y_min, w, h] boxes_xywh = row['bbox'].tolist() # преобразуем в [x_min, y_min, x_max, y_max] для albumentations и torchmetrics boxes_xyxy = [[x, y, x + w, y + h] for (x, y, w, h) in boxes_xywh] if self.transform is not None: transformed = self.transform(image=image, bboxes=boxes_xyxy, labels=labels) image, boxes_xyxy, labels = transformed["image"], transformed["bboxes"], transformed["labels"] else: image = transforms.ToTensor()(image) target['boxes'] = torch.tensor(np.array(boxes_xyxy), dtype=torch.float32) target['labels'] = torch.tensor(labels, dtype=torch.int64) return image, target def collate_fn(batch): batch = tuple(zip(*batch)) images = torch.stack(batch[0]) return images, batch[1]</pre>

Чтобы модель не переобучалась, можно добавить больше аугментаций, весь список можно посмотреть тут [\[ссылка\]](#).

Какие можно использовать аугментации?

- Добавить зум `RandomResizedCrop`,
- Сделать цветовые аугментации типа `RandomBrightnessContrast` и/или `HueSaturationValue`,
- Добавить шум `GaussNoise`,
- Вырезать случайные части изображения `CoarseDropout`,
- И любые другие!

Аугментации можно комбинировать посредством `A.OneOf`, `A.SomeOf` или `A.RandomOrder`.

Хоть аугментации ограничиваются только вашей фантазией, перед обучением советуем посмотреть на результат преобразований и убедиться, что изображение ещё поддается детекции:)

```
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)

#train_transform = A.Compose([
#    A.Resize(800, 800),
#    A.HorizontalFlip(p=0.5),
#    A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.15, rotate_limit=10, p=0.5),
#    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.4),
#    A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20, val_shift_limit=10, p=0.2),
#    A.RandomGamma(p=0.3),
#    A.Normalize(mean=mean, std=std),
#    ToTensorV2(),
#], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

train_transform = A.Compose([
    [
        A.Resize(640, 640),
        A.Normalize(mean=mean, std=std),
        # HorizontalFlip меняет и ббоксы!
        A.HorizontalFlip(p=0.5),
        A.RandomBrightnessContrast(p=0.2),
        ToTensorV2(),
    ],bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

test_transform = A.Compose([
    A.Resize(640, 640),
    A.Normalize(mean=mean, std=std),
    ToTensorV2(),
], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))
```

```
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)

train_transform = A.Compose([
    A.Resize(800, 800),
    A.RandomResizedCrop(size=(800, 800), scale=(0.5, 1.0), p=0.5),
    A.HorizontalFlip(p=0.5),
    A.Transpose(p=0.1),
    A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.15, rotate_limit=10, p=0.5),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.4),
    A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20, val_shift_limit=10, p=0.3),
    A.RGBShift(r_shift_limit=10, g_shift_limit=10, b_shift_limit=10, p=0.3),
    A.RandomGamma(p=0.3),
    A.ISONoise(p=0.2),
    A.MotionBlur(blur_limit=3, p=0.1),
    A.Normalize(mean=mean, std=std),
    ToTensorV2(),
], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

test_transform = A.Compose([
    A.Resize(800, 800),
    A.Normalize(mean=mean, std=std),
    ToTensorV2(),
], bbox_params=A.BboxParams(format='pascal_voc', label_fields=['labels']))

/usr/local/lib/python3.12/dist-packages/albumentations/core/validation.py:114: UserWarning: ShiftScaleRotate is a special case of Affine transform. Please use Affine transform instead.
  original_init(self, **validated_kwargs)
```

Не забываем инициализировать наш датасет

```
train_dataset = HaloDataset(df_train, transform=train_transform)
test_dataset = HaloDataset(df_test, transform=test_transform)
```

▼ Переделываем модель [4 балла]

В семинаре мы реализовали самый базовый детектор, а сейчас настало время его улучшать.

Backbone [1 балл]

Хорошей практикой считается размораживать несколько последних слоев в backbone, это позволяет немного улучшить качество модели. Давайте улучшим класс Backbone из лекции, добавив ему возможность разморозки **k** последних слоев или блоков (на ваш выбор).

```
import timm
timm.list_models(pretrained=True)
```

[Показать скрытые выходные данные](#)

```
model_name = 'convnext_base.fb_in22k_ft_in1k_384' #convnext_large.fb_in22k_ft_in1k_384
```

```
timm.get_pretrained_cfg(model_name)

PretrainedCfg(url='https://dl.fbaipublicfiles.com/convnext/convnext_base_22k_1k_384.pth', file=None, state_dict=None, hf_hub_id='timm/convnext_base.fb_in22k_ft_in1k_384', hf_hub_filename=None, source=None, architecture='convnext_base', tag='fb_in22k_ft_in1k_384', custom_load=False, input_size=(3, 384, 384), test_input_size=None, min_input_size=None, fixed_input_size=False, interpolation='bicubic', crop_pct=1.0, test_crop_pct=None, crop_mode='squash', mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225), num_classes=1000, label_offset=None, label_names=None, label_descriptions=None, pool_size=(12, 12), test_pool_size=None, first_conv='stem.0', classifier='head.fc', license='apache-2.0', description=None, origin_url=None, paper_name=None, paper_ids=None, notes=None)
```

```
import timm
model = timm.create_model(model_name, pretrained=True, features_only=True, out_indices=[-1,-2,-3,-4])
```

```
x = torch.randn(1, 3, 384, 384)
with torch.no_grad():
    features = model(x)
    for i, f in enumerate(features):
        print(i, f.shape)

0 torch.Size([1, 128, 96, 96])
1 torch.Size([1, 256, 48, 48])
2 torch.Size([1, 512, 24, 24])
3 torch.Size([1, 1024, 12, 12])
```

```
layers = list(model.named_modules())
for i in layers[-3:]:
    print(i)
```

```
('stages_3.blocks.2.mlp.drop2', Dropout(p=0.0, inplace=False))
('stages_3.blocks.2.shortcut', Identity())
('stages_3.blocks.2.drop_path', Identity())
```

```
model.feature_info.channels()
```

```
[1024, 512, 256, 128]
```

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.

```
from torchinfo import summary
a = summary(model)
a
```

Layer (type:depth-idx)	Param #
FeatureListNet	--
└Conv2d: 1-1	6,272
└LayerNorm2d: 1-2	256
└ConvNextStage: 1-3	--
└Identity: 2-1	--
└Sequential: 2-2	--
└ConvNextBlock: 3-1	138,496
└ConvNextBlock: 3-2	138,496
└ConvNextBlock: 3-3	138,496
└ConvNextStage: 1-4	--
└Sequential: 2-3	--
└LayerNorm2d: 3-4	256
└Conv2d: 3-5	131,328
└Sequential: 2-4	--
└ConvNextBlock: 3-6	539,136
└ConvNextBlock: 3-7	539,136
└ConvNextBlock: 3-8	539,136
└ConvNextStage: 1-5	--
└Sequential: 2-5	--
└LayerNorm2d: 3-9	512
└Conv2d: 3-10	524,800
└Sequential: 2-6	--
└ConvNextBlock: 3-11	2,126,848
└ConvNextBlock: 3-12	2,126,848
└ConvNextBlock: 3-13	2,126,848
└ConvNextBlock: 3-14	2,126,848
└ConvNextBlock: 3-15	2,126,848
└ConvNextBlock: 3-16	2,126,848
└ConvNextBlock: 3-17	2,126,848
└ConvNextBlock: 3-18	2,126,848
└ConvNextBlock: 3-19	2,126,848
└ConvNextBlock: 3-20	2,126,848
└ConvNextBlock: 3-21	2,126,848
└ConvNextBlock: 3-22	2,126,848
└ConvNextBlock: 3-23	2,126,848
└ConvNextBlock: 3-24	2,126,848
└ConvNextBlock: 3-25	2,126,848
└ConvNextBlock: 3-26	2,126,848
└ConvNextBlock: 3-27	2,126,848
└ConvNextBlock: 3-28	2,126,848
└ConvNextBlock: 3-29	2,126,848
└ConvNextBlock: 3-30	2,126,848
└ConvNextBlock: 3-31	2,126,848
└ConvNextBlock: 3-32	2,126,848
└ConvNextBlock: 3-33	2,126,848
└ConvNextBlock: 3-34	2,126,848
└ConvNextBlock: 3-35	2,126,848
└ConvNextBlock: 3-36	2,126,848
└ConvNextBlock: 3-37	2,126,848
└ConvNextStage: 1-6	--
└Sequential: 2-7	--
└LayerNorm2d: 3-38	1,024
└Conv2d: 3-39	2,098,176
└Sequential: 2-8	--
└ConvNextBlock: 3-40	8,448,000

```
from torch import nn
class Backbone(nn.Module):
    def __init__(self, model_name = model_name, unfreeze_last = 0, out_indices = [-1, -2, -3, -4]):
        super().__init__()
        self.backbone = timm.create_model(model_name, pretrained=True, features_only=True, out_indices=out_indices)

        for param in self.backbone.parameters():
            param.requires_grad = False

        if unfreeze_last > 0:
            lays = list(self.backbone.children())
            for l in lays[-unfreeze_last:]:
                for p in l.parameters():
                    p.requires_grad = True

    def forward(self, x):
        return self.backbone(x)
```

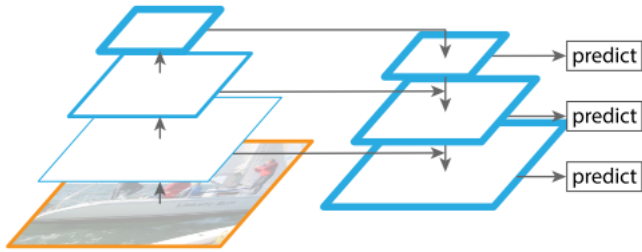
```
backbone = Backbone()
x = torch.randn(1, 3, 384, 384)
with torch.no_grad():
    features = backbone(x)
    for i, f in enumerate(features):
        print(i, f.shape)
```

```
0 torch.Size([1, 128, 96, 96])
1 torch.Size([1, 256, 48, 48])
2 torch.Size([1, 512, 24, 24])
3 torch.Size([1, 1024, 12, 12])
```

▼ NECK [2 балла]

Следующее улучшение коснется шеи. Предлагаем реализовать знакомую из лекции архитектуру FPN.

Feature Pyramid Network



(d) Feature Pyramid Network

Она состоит из top-down пути, в котором происходит 2 вещи:

1. Увеличивается пространственная размерность фичей,
2. С помощью skipконнекшенов, добавляются фичи из backbone модели.

Для увеличения пространственной размерности используется **nearest neighbor upsampling**, а фичи из шеи и бекбоуна суммируются.

TIPS:

- Можете использовать базовые классы из лекции,
- Воспользуйтесь AnchorGenerator-ом, чтобы создавать якоря сразу для нескольких выходов,
- Не забудьте использовать nn.ModuleList, если захотите сделать динамическое количество голов у модели,
- Также, можно добавить доп конволюцию (3x3 с паддингом) у каждого выхода шеи.

```
model.feature_info.channels()

[1024, 512, 256, 128]

import torch.nn.functional as F
class SimplifiedFPN(nn.Module):
    """Очень простая шея: один слой conv + batch_norm + activation. """
    def __init__(self, out_channels, backbone):
        super().__init__()
        self.backbone = backbone
        in_channels = backbone.feature_info.channels()

        self.lateral_convs = nn.ModuleList([
            nn.Conv2d(in_ch, out_channels, 1) for in_ch in in_channels
        ])
        self.fpn_convs = nn.ModuleList([
            nn.Conv2d(out_channels, out_channels, 3, padding=1)
            for _ in range(len(in_channels))
        ])
        self.bns = nn.ModuleList([
            nn.BatchNorm2d(out_channels) for _ in range(len(in_channels))
        ])

    def forward(self, features):
        features = features[::-1]
        laterals = [conv(f) for conv, f in zip(self.lateral_convs, features)]

        x = self.fpn_convs[0](laterals[0])
        x = self.bns[0](x)
        x = F.relu(x)
        outputs = [x]

        for i in range(1, len(laterals)):
            x = F.interpolate(x, size=laterals[i].shape[2:], mode='nearest')
            x = x + laterals[i]
            x = self.fpn_convs[i](x)
            x = self.bns[i](x)
            x = F.relu(x)
            outputs.append(x)

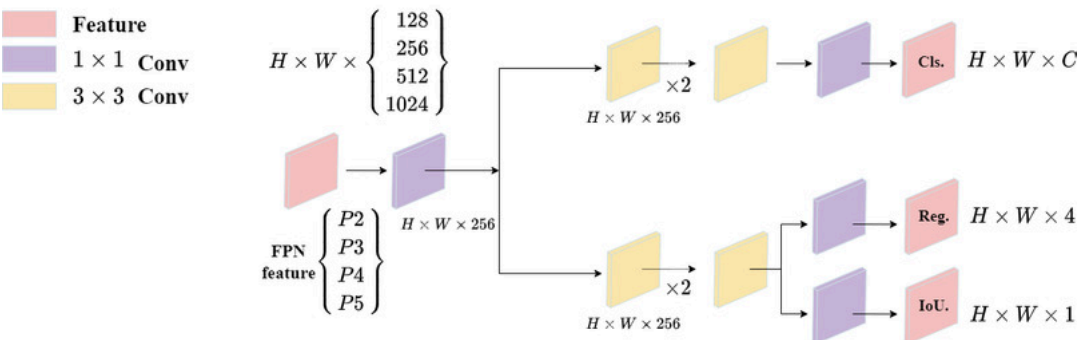
        return outputs
```

▼ Head [1 балл]

В качестве головы можно выбрать **один из двух** вариантов:

1. Decoupled Head

Реализовать Decoupled Head из [YOLOX](#).



TIP: Возьмите за основу голову из семинара, тк она сильно похожа на Decoupled Head.

Изменять количество параметров у шей на разных уровнях не обязательно.

2. Confidence score free head

Нужно взять за основу голову из семинара и полностью убрать предсказание confidence score. Чтобы модель предсказывала только 2 группы: ббоксы и классы.

Есть следующие способы удаления confidence score:

- Добавление нового класса ФОН. Обычно его обозначают нулевым классом.
- Присваивание ббоксам БЕЗ объекта вектор из нулей в качестве таргета.

Выберете тот, который вам больше нравится и будьте внимательны при расчете лосса!

Важно! Удаление confidence score повлияет на следующие методы из семинара:

- target_assign
- ComputeLoss
- _filter_predictions

```
class Head(nn.Module):
    def __init__(self, in_channels, num_anchors, num_classes, c=256):
        super().__init__()
        self.conv = nn.Conv2d(in_channels, in_channels, kernel_size=3, padding=1)

        self.cls_pr = nn.Sequential(
            nn.Conv2d(in_channels, c, kernel_size=3, padding=1),
            nn.BatchNorm2d(c),
            nn.ReLU(inplace=True)
        )
        self.cls_head = nn.Conv2d(c, num_anchors * (num_classes + 1), kernel_size=1)

        self.reg_pr = nn.Sequential(
```

```
nn.Conv2d(in_channels, c, kernel_size=3, padding=1),
nn.BatchNorm2d(c),
nn.ReLU(inplace=True)
)
self.reg_head = nn.Conv2d(c, num_anchors * 4, kernel_size=1)
self.num_classes = num_classes

def forward(self, x):
    x = F.relu(self.conv(x))
    cls = self.cls_pr(x)
    reg = self.reg_pr(x)
    cls_logits = self.cls_head(cls)
    bbox_preds = self.reg_head(reg)
    return cls_logits, bbox_preds
```

Теперь можно снова реализовать класс детектора с учетом всех частей выше!

```
class Detector(nn.Module):
    def __init__(self,
                  backbone_model_name=model_name,
                  neck_n_channels=256,
                  num_classes=2,
                  anchor_sizes=((32,), (64,), (128,), (256,)),
                  anchor_ratios=(0.5, 1.0, 2.0),
                  input_size=(640,640),
                  unfreeze_last=0):
        super().__init__()
        self.num_classes = num_classes
        self.backbone = Backbone(backbone_model_name, unfreeze_last,
                                out_indices=[-1, -2, -3, -4])
        self.neck = SimplifiedFPN(out_channels=neck_n_channels,
                                  backbone=self.backbone.backbone)

        self.num_anchors_per_loc = len(anchor_sizes[0]) * len(anchor_ratios)
        self.head = Head(in_channels=neck_n_channels,
                          num_anchors=self.num_anchors_per_loc,
                          num_classes=num_classes)

        reductions = self.backbone.backbone.feature_info.reduction()
        grid_sizes = []
        strides = []
        for r in reductions:
            gs_h = input_size[0] // r
            gs_w = input_size[1] // r
            grid_sizes.append([gs_h, gs_w])
            strides.append([r, r])

        anchor_gen = AnchorGenerator(sizes=anchor_sizes,
                                     aspect_ratios=(anchor_ratios,) * len(reductions))
        anchors_per_level = anchor_gen.grid_anchors(grid_sizes, strides=strides)

        #anchors_per_level = anchors_per_level[::-1]

        anchors = torch.cat(anchors_per_level, dim=0).unsqueeze(0)
        anchor_centers = (anchors[:, :, :2] + anchors[:, :, 2:]) / 2
        anchor_sizes_wh = anchors[:, :, 2:] - anchors[:, :, :2]

        self.register_buffer("anchors", anchors)
        self.register_buffer("anchor_centers", anchor_centers)
        self.register_buffer("anchor_sizes", anchor_sizes_wh)
        self.num_anchors_per_level = [a.shape[0] for a in anchors_per_level]

    def forward(self, x):
        features = self.backbone(x)
        neck_features = self.neck(features)

        head_outputs = [self.head(f) for f in neck_features]

        all_cls_logits = []
        all_bbox_preds = []
        for cls_logits_level, bbox_preds_level in head_outputs:
            N, _, H, W = cls_logits_level.shape
            cls_logits_level = cls_logits_level.permute(0, 2, 3, 1).contiguous()
            cls_logits_level = cls_logits_level.view(N, -1, self.num_classes + 1)
            bbox_preds_level = bbox_preds_level.permute(0, 2, 3, 1).contiguous()
            bbox_preds_level = bbox_preds_level.view(N, -1, 4)
            all_cls_logits.append(cls_logits_level)
            all_bbox_preds.append(bbox_preds_level)

        cls_logits = torch.cat(all_cls_logits, dim=1)
        bbox_offsets = torch.cat(all_bbox_preds, dim=1)

        if self.training:
            return bbox_offsets, cls_logits

        cls_probs = torch.softmax(cls_logits, dim=-1)
        confidence = 1 - cls_probs[:, :, 0]
        bboxes_xyxy = self.decode_bboxes(bbox_offsets)
        return bboxes_xyxy, confidence, cls_probs

    def decode_bboxes(self, bbox_offsets):
        """Декодирование без sigmoid – прямая регрессия."""
        tx = bbox_offsets[:, :, 0]
        ty = bbox_offsets[:, :, 1]
        tw = bbox_offsets[:, :, 2]
        th = bbox_offsets[:, :, 3]

        anchor_cx = self.anchor_centers[:, :, 0]
        anchor_cy = self.anchor_centers[:, :, 1]
        anchor_w = self.anchor_sizes[:, :, 0]
        anchor_h = self.anchor_sizes[:, :, 1]

        pred_cx = anchor_cx + tx * anchor_w
        pred_cy = anchor_cy + ty * anchor_h
        pred_w = torch.exp(tw) * anchor_w
        pred_h = torch.exp(th) * anchor_h

        x1 = pred_cx - pred_w / 2
        y1 = pred_cy - pred_h / 2
        x2 = pred_cx + pred_w / 2
        y2 = pred_cy + pred_h / 2
        return torch.stack([x1, y1, x2, y2], dim=-1)
```


В этой секции предлагается заменить функцию `assign_target` на более современный алгоритм который называется Task alignment learning.

Он описан в статье [TOOD](#) в секции 3.2. Для удобства вот его основные шаги:

1. Посчитать значение метрики для каждого предсказанного ббокса:

$$t = s^\alpha * u^\beta$$

где,

- s — classification score, или вероятность принадлежности предсказанного ббокса к классу реального ббокса (**GT**);
- u — IoU между предсказанным и реальным ббоксами;
- α, β — нормализационные константы, обычно $\alpha = 6.0, \beta = 1.0$.

2. Отфильтровать предсказания на основе **GT**.

Для якорных детекторов, обычно, выбираются только те предсказания, центры якорей которых находятся внутри GT.

3. Для каждого **GT** выбрать несколько (обычно 5 или 13) самых подходящих предсказаний.

4. Если предсказание рассматривается в качестве подходящего для нескольких **GT** — выбрать **GT** с наибольшим пересечением по IoU.

ВАЖНО: если будете использовать Runner из лекции, не забудьте поменять параметры в `self.assign_target_method` в методе `_run_train_epoch`.

```
def safe_logit(x):
    eps = 1e-6
    x = torch.clamp(x, eps, 1 - eps)
    return torch.log(x / (1 - x))

def get_target_offset(anchor_box, gt_box):
    """Расчёт таргет-смещений от якоря до GT (хуху)."""
    gt_center = (gt_box[:2] + gt_box[2:]) / 2
    gt_size = gt_box[2:] - gt_box[:2]
    anchor_center = (anchor_box[:2] + anchor_box[2:]) / 2
    anchor_size = anchor_box[2:] - anchor_box[:2]

    tx = (gt_center[0] - anchor_center[0]) / anchor_size[0]
    ty = (gt_center[1] - anchor_center[1]) / anchor_size[1]
    target_tw = torch.log(gt_size[0] / anchor_size[0])
    target_th = torch.log(gt_size[1] / anchor_size[1])
    return torch.tensor([tx, ty, target_tw, target_th], device=anchor_box.device)
```

```
def assign_target_tal(cls_logits, bbox_preds, anchors, gt_boxes, gt_labels,
                     num_classes, alpha=6.0, beta=1.0, top_k=5):
    """
    cls_logits: [N_anchors, num_classes+1]
    bbox_preds: [N_anchors, 4] (tx, ty, tw, th) — без sigmoid в формуле
    anchors:    [N_anchors, 4] (x1,y1,x2,y2)
    gt_boxes:   [N_gt, 4]      (x1,y1,x2,y2)
    gt_labels:  [N_gt]        0..num_classes-1
    """
    device = anchors.device
    num_anchors = anchors.shape[0]
    num_gt = gt_boxes.shape[0]

    target_offsets = torch.zeros(num_anchors, 4, device=device)
    target_cls = torch.zeros(num_anchors, dtype=torch.long, device=device) # 0 = фон

    if num_gt == 0:
        return target_offsets, target_cls

    anchor_w = anchors[:, 2] - anchors[:, 0]
    anchor_h = anchors[:, 3] - anchors[:, 1]
    anchor_cx = (anchors[:, 0] + anchors[:, 2]) / 2
    anchor_cy = (anchors[:, 1] + anchors[:, 3]) / 2

    tx, ty, tw, th = bbox_preds[:, 0], bbox_preds[:, 1], bbox_preds[:, 2], bbox_preds[:, 3]
    pred_cx = anchor_cx + tx * anchor_w
    pred_cy = anchor_cy + ty * anchor_h
    pred_w = torch.exp(tw) * anchor_w
    pred_h = torch.exp(th) * anchor_h
    pred_x1 = pred_cx - pred_w / 2
    pred_y1 = pred_cy - pred_h / 2
    pred_x2 = pred_cx + pred_w / 2
    pred_y2 = pred_cy + pred_h / 2
    pred_boxes = torch.stack([pred_x1, pred_y1, pred_x2, pred_y2], dim=1)

    cls_probs = torch.softmax(cls_logits, dim=-1) # [N_anchors, num_classes+1]
    ious = box_iou(pred_boxes, gt_boxes) # [N_anchors, N_gt]

    inside_masks = []
    for gt_idx in range(num_gt):
        gt = gt_boxes[gt_idx]
        inside = (anchor_cx >= gt[0]) & (anchor_cx <= gt[2]) & \
            (anchor_cy >= gt[1]) & (anchor_cy <= gt[3])
        inside_masks.append(inside)
    inside_mask = torch.stack(inside_masks, dim=1)

    t = torch.zeros_like(ious)
    for gt_idx in range(num_gt):
        gt_label = gt_labels[gt_idx].item()
        s = cls_probs[:, gt_label + 1]
        t[:, gt_idx] = s.pow(alpha) * ious[:, gt_idx].pow(beta)
    t[~inside_mask] = float('-inf')

    assigned_gt = -torch.ones(num_anchors, dtype=torch.long, device=device)
    best_iou_for_anchor = torch.zeros(num_anchors, device=device)

    for gt_idx in range(num_gt):
        k = min(top_k, num_anchors)
        values, top_indices = torch.topk(t[:, gt_idx], k=k)
        for idx in top_indices:
            if assigned_gt[idx] == -1 or ious[idx, gt_idx] > best_iou_for_anchor[idx]:
                assigned_gt[idx] = gt_idx
                best_iou_for_anchor[idx] = ious[idx, gt_idx]

    pos_indices = (assigned_gt != -1).nonzero(as_tuple=True)[0]
    for pos_idx in pos_indices:
        gt_idx = assigned_gt[pos_idx]
        target_offsets[pos_idx] = get_target_offset(anchors[pos_idx], gt_boxes[gt_idx])
        target_cls[pos_idx] = gt_labels[gt_idx].item() + 1 # +1, т.к. 0 — фон

    return target_offsets, target_cls
```

Вместо SmoothL1, который используется в семинаре, реализуем лосс, основанный на пересечении ббоксов. В качестве тренировки давайте напишем Distance Intersection over Union (DIoU).



Для его реализации разобьем задачу на части:

1. Реализуем IoU:

Пусть даны координаты для предсказанного (B^p) и истинного (B^g) ббоксов в формате XYYX или VOC PASCAL (левый верхний и правый нижний углы):

$B^p = (x_1^p, y_1^p, x_2^p, y_2^p)$, $B^g = (x_1^g, y_1^g, x_2^g, y_2^g)$, тогда алгоритм расчета будет следующий:

1. Найдем площади обоих ббоксов:

$$A^p = (x_2^p - x_1^p) * (y_2^p - y_1^p)$$
$$A^g = (x_2^g - x_1^g) * (y_2^g - y_1^g)$$

2. Посчитаем пересечение между ббоксами:

Тут мы предлагаем вам подумать как в общем виде можно рассчитать размеры ббокса, который будет являться пересечением B^p и B^g , а затем посчитать его площадь:

$$x_1^I = \max(x_1^p, x_1^g)$$
$$x_2^I = \min(x_2^p, x_2^g)$$

В общем виде, площадь будет записываться следующим образом:

Если $x_2^I > x_1^I$ & $y_2^I > y_1^I$, тогда:

$$I = (x_2^I - x_1^I) * (y_2^I - y_1^I)$$

Иначе, $I = 0$.

3. Считаем объединение ббоксов.

Мы можем посчитать эту площадь как сумму площадей двух ббоксов минус площадь пересечения (тк мы считаем её два раз в сумме площадей):

$$U = A^p + A^g - I$$

4. Вычисляем IoU.

$$IoU = \frac{I}{U}$$

2. Посчитаем диагональ выпуклой оболочки:

Для расчета диагонали, сначала выпишите координаты верхнего левого и правого нижнего углов. Подумайте, чему будут равны эти координаты в общем случае?

$$x_1^c = \max(x_1^p, x_1^g)$$
$$x_2^c = \min(x_2^p, x_2^g)$$

Подсказка: Нарисуйте несколько вариантов пересечений предсказания и GT на бумажке, и выпишите координаты для выпуклой оболочки.

Тогда квадрат диагонали можно посчитать по формуле:

$$c^2 = (x_2^c - x_1^c)^2 + (y_2^c - y_1^c)^2$$

3. Рассчитаем расстояние между цетрами ббоксов:

Сначала находим координаты центров каждого из ббоксов (если ббоксы в формате YOLO, то и считать ничего не нужно), затем считаем Евклидово расстояние между центрами.

$d =$

Собираем все части вместе и считаем лосс по формуле:

$$DIoU = 1 - IoU + \frac{d^2}{c^2}$$

Помните, что пар ббоксов может быть много! Возвращайте усредненное значение лосса.

```
from torchvision.ops import distance_box_iou_loss
```

```
def gen_bbox(num_boxes=10):
    min_corner = torch.randint(0, 100, (num_boxes, 2))
    max_corner = torch.randint(50, 150, (num_boxes, 2))

    for i in range(2):
        wrong_order = min_corner[:, i] > max_corner[:, i]
        if wrong_order.any():
            min_corner[wrong_order, i], max_corner[wrong_order, i] = max_corner[wrong_order, i], min_corner[wrong_order, i]
    return torch.cat((min_corner, max_corner), dim=1)
```

```
pred_boxes = gen_bbox(num_boxes=100)
true_boxes = gen_bbox(num_boxes=100)
```

```
print(f" DIoU: {distance_box_iou_loss(pred_boxes, true_boxes, reduction='mean').item()}")
```

```
DIoU: 1.0079089403152466
```

```
def diou_loss(pred_boxes, gt_boxes, eps=1e-7):
    area_p = (pred_boxes[:, 2] - pred_boxes[:, 0]) * (pred_boxes[:, 3] - pred_boxes[:, 1])
    area_g = (gt_boxes[:, 2] - gt_boxes[:, 0]) * (gt_boxes[:, 3] - gt_boxes[:, 1])

    x1_i = torch.max(pred_boxes[:, 0], gt_boxes[:, 0])
    y1_i = torch.max(pred_boxes[:, 1], gt_boxes[:, 1])
    x2_i = torch.min(pred_boxes[:, 2], gt_boxes[:, 2])
    y2_i = torch.min(pred_boxes[:, 3], gt_boxes[:, 3])

    inter_w = (x2_i - x1_i).clamp(min=0)
    inter_h = (y2_i - y1_i).clamp(min=0)
    inter = inter_w * inter_h

    union = area_p + area_g - inter
    iou = inter / (union + eps)

    c_p_x = (pred_boxes[:, 0] + pred_boxes[:, 2]) / 2
    c_p_y = (pred_boxes[:, 1] + pred_boxes[:, 3]) / 2
    c_g_x = (gt_boxes[:, 0] + gt_boxes[:, 2]) / 2
    c_g_y = (gt_boxes[:, 1] + gt_boxes[:, 3]) / 2

    d2 = (c_p_x - c_g_x) ** 2 + (c_p_y - c_g_y) ** 2
```



```
x1_c = torch.min(pred_boxes[:, 0], gt_boxes[:, 0])
y1_c = torch.min(pred_boxes[:, 1], gt_boxes[:, 1])
x2_c = torch.max(pred_boxes[:, 2], gt_boxes[:, 2])
y2_c = torch.max(pred_boxes[:, 3], gt_boxes[:, 3])

c2 = (x2_c - x1_c) ** 2 + (y2_c - y1_c) ** 2

diou = 1 - iou + d2 / (c2 + eps)

return diou.mean()
```

```
import numpy as np
pred_boxes = gen_bbox(num_boxes=1000)
true_boxes = gen_bbox(num_boxes=1000)

# проверим что написанный лосс выдает те же результаты что и лосс из торча.
assert np.isclose(diou_loss(pred_boxes, true_boxes), distance_box_iou_loss(pred_boxes, true_boxes, reduction="mean"))
```

Кто больше? [5 баллов]

Наконец то мы дошли до самый интересной части. Тут мы раздаем очки за mAP'ы!

Все что вы написали выше вам поможет улучшить качество итогового детектора, настало время узнать насколько сильно :)

За достижения порога по mAP на тестовом наборе вы получаете баллы:

- 0.05 mAP [1]
- 0.1 mAP [2]
- 0.2 mAP [5]

- TIPS:**
1. На семинаре мы специально не унифицировали формат ббоксов между методами, чтобы обратить ваше внимание что за этим нужно следить. Чтобы было проще, сразу унифицируете формат по всему ноутбуку. Советуем использовать формат хуху, тк IoU и NMS из torch используют именно этот формат. (Не забудьте поменять формат у таргета в [HaloDataset](#)).
 2. Попробуйте перейти к IoU-based лоссу при обучении. То есть обучать не смещения, а сразу предсказывать ббокс.
 3. Поэкспериментируйте с подходами target assignment'а в процессе обучения. Например, можно на первых итерациях использовать обычный метод, а затем подключить TAL.
 4. Добавьте аугментаций!

Можно взять [albugmentations](#), библиотеку, которую мы использовали всеминаре. Или базовые аугментации из торча [тык](#). Если будете использовать торч, не забудьте про ббоксы, transforms из коробки не будет их агументировать.

5. Можете реализовать другую шею, которую мы обсуждали на лекции [Path Aggregation Network](#) она точно улучшит ваше итоговое качество.
6. Попробуйте добавлять различные блоки из YOLO архитектур в шею вместо единичных конволюционных слоев. (Например, замените конволюции 3x3 на CSP блоки).
7. Попробуйте заменить NMS на другой метод (WeightedNMS, SoftNMS, etc.). Немного ссылок:
 - Статья про SoftNMS [тык](#)
 - Статья про WeightedNMS [тык](#)
 - Есть их реализация, правда на нумбе [git](#)
8. Не бойтесь экспериментировать и удачи!

Также, напишите развернутые ответы на следующие вопросы:

- Questions:**
1. Какой метод label assignment'а помогает лучше обучаться модели? Почему?
 2. Какое из сделанных вами улучшений внесло наибольший вклад в качество модели? Как вы думаете, почему это произошло?
 3. Какое из сделанных вами улучшений вообще не изменило метрику? Как вы думаете, почему это произошло?

```
import torch
import numpy as np
from tqdm import tqdm
from functools import partial
from torchvision.ops import batched_nms
from torchmetrics.detection import MeanAveragePrecision

class Runner:

    def __init__(self, model, compute_loss, optimizer, train_dataloader, assign_target_method, device=None,
                 scheduler=None, assign_target_kwargs=None,
                 val_dataloader=None, val_every=5, score_threshold=0.1, nms_threshold=0.5):
        self.model = model
        self.compute_loss = compute_loss
        self.optimizer = optimizer
        self.train_dataloader = train_dataloader
        assign_target_kwargs = {} if assign_target_kwargs is None else assign_target_kwargs
        self.assign_target_method = partial(assign_target_method, **assign_target_kwargs)
        self.device = "cpu" if device is None else device
        self.scheduler = scheduler

        # Валидационные параметры
        self.val_dataloader = val_dataloader
        self.val_every = val_every
        self.score_threshold = score_threshold
        self.nms_threshold = nms_threshold

        # Хранение истории
        self.val_loss_history = []
        self.batch_loss = []
        self.epoch_loss = []
        self.val_metric = []

    def _run_train_epoch(self, dataloader, verbose=True):
        self.model.train()
        batch_loss = []
        for images, targets in (pbar := tqdm(dataloader, desc="Process train epoch", leave=False)):
            images = images.to(self.device)
            # Модель возвращает bbox_preds, cls_logits
            bbox_preds, cls_logits = self.model(images)
            anchors = self.model.anchors.view(-1, 4) # [total_anchors, 4]
            accum_loss = 0.0
            for ix in range(images.shape[0]):
                gt_boxes = targets[ix]['boxes'].to(self.device)
                gt_labels = targets[ix]['labels'].to(self.device)

                pred_bbox_i = bbox_preds[ix] # [total_anchors, 4]
                pred_cls_i = cls_logits[ix] # [total_anchors, num_classes+1]

            # TAL assign
```

```

        target_offsets, target_cls = self.assign_target_method(
            pred_cls_i, pred_bbox_i, anchors,
            gt_boxes, gt_labels,
            num_classes=self.model.num_classes
        )

        outputs_ix = (pred_bbox_i, pred_cls_i)
        loss = self.compute_loss(outputs_ix, (target_offsets, target_cls), anchors)
        accum_loss += loss

    accum_loss = accum_loss / images.shape[0]
    batch_loss.append(accum_loss.cpu().detach().item())

    self.optimizer.zero_grad()
    accum_loss.backward()
    self.optimizer.step()

    return batch_loss

@torch.no_grad()
def _filter_predictions(self, outputs, score_threshold, nms_threshold):
    bboxes, confidence, cls_probs = outputs
    preds = []
    for i in range(len(bboxes)):
        obj_probs = cls_probs[i][:, 1:]
        best_scores, best_idx = obj_probs.max(dim=1)
        final_scores = confidence[i] * best_scores

    mask = final_scores > score_threshold
    if not mask.any():
        preds.append({
            'boxes': torch.empty(0, 4),
            'scores': torch.empty(0),
            'labels': torch.empty(0, dtype=torch.long)
        })
        continue

    boxes_i = bboxes[i][mask]
    scores_i = final_scores[mask]
    labels_i = best_idx[mask]

    keep = batched_nms(boxes_i, scores_i, labels_i, nms_threshold)
    preds.append({
        'boxes': boxes_i[keep].cpu(),
        'scores': scores_i[keep].cpu(),
        'labels': labels_i[keep].cpu()
    })
    return preds

@torch.no_grad()
def validate(self):
    self.model.eval()
    total_loss = 0.0
    metric = MeanAveragePrecision(box_format="xyxy", iou_type="bbox")
    anchors = self.model.anchors.view(-1, 4)

    for images, targets in tqdm(self.val_dataloader, desc="Val loss + mAP", leave=False):
        images = images.to(self.device)

        training_flag = self.model.training
        self.model.train()
        bbox_preds, cls_logits = self.model(images)
        self.model.train(training_flag)

        batch_loss = 0.0
        for i in range(images.shape[0]):
            gt_boxes = targets[i]['boxes'].to(self.device)
            gt_labels = targets[i]['labels'].to(self.device)
            target_offsets, target_cls = self.assign_target_method(
                cls_logits[i], bbox_preds[i], anchors,
                gt_boxes, gt_labels,
                num_classes=self.model.num_classes
            )
            loss = self.compute_loss(
                (bbox_preds[i], cls_logits[i]),
                (target_offsets, target_cls),
                anchors
            )
            batch_loss += loss
        total_loss += (batch_loss / images.shape[0]).item()
    self.model.eval()
    eval_outputs = self.model(images)
    preds = self._filter_predictions(eval_outputs, self.score_threshold, self.nms_threshold)
    metric.update(preds, targets)

    avg_loss = total_loss / len(self.val_dataloader)
    mAP = metric.compute()['map'].item()
    return avg_loss, mAP

def train(self, num_epochs=10, verbose=True):
    val_desc = ""
    for epoch in (epoch_pbar := tqdm(range(1, num_epochs+1), desc="Train epoch", total=num_epochs)):
        loss = self._run_train_epoch(self.train_dataloader, verbose=verbose)
        self.batch_loss.extend(loss)
        self.epoch_loss.append(np.mean(self.batch_loss[-len(self.train_dataloader):]))

        if self.val_dataloader is not None and epoch % self.val_every == 0:
            val_loss, val_map = self.validate()
            self.val_metric.append(val_map)
            self.val_loss_history.append(val_loss)
            val_desc = f" Val loss: {val_loss:.4f} mAP: {val_map:.4f}"

        if verbose:
            epoch_pbar.set_description(
                f"Epoch {epoch} loss: Train {self.epoch_loss[-1]:.4f}" + val_desc
            )
        if self.scheduler is not None:
            self.scheduler.step()

```

```

from torch.utils.data import Dataset
from torch.utils.data import DataLoader
train_dataloader = DataLoader(train_dataset, batch_size=8, shuffle=True, collate_fn=collate_fn)
test_dataloader = DataLoader(test_dataset, batch_size=4, shuffle=False, collate_fn=collate_fn)

```

```
from torchvision.ops import box_iou
```

```
def iou_loss(pred_boxes, gt_boxes, eps=1e-7):
```

```
    iou = box_iou(pred_boxes, gt_boxes)
    diag = torch.diag(iou)
    return (1 - diag).mean()
```

```
import torch
import torch.nn as nn
from torchvision.ops import distance_box_iou_loss
class ComputeLoss:
    def __init__(self, bbox_loss=None, cls_loss=None, weight_bbox=5, weight_cls=1):
        self.bbox_loss = bbox_loss if bbox_loss is not None else distance_box_iou_loss#####
        self.cls_loss = nn.CrossEntropyLoss(reduction='none')
        self.weight_bbox = weight_bbox
        self.weight_cls = weight_cls

    def __call__(self, predicts, targets, anchors):
        pred_offsets, pred_cls_logits = predicts
        target_offsets, target_cls = targets

        ce = self.cls_loss(pred_cls_logits, target_cls)
        pos_mask = target_cls > 0
        num_pos = pos_mask.sum().float().clamp(min=1.0)
        num_neg = (~pos_mask).sum().float().clamp(min=1.0)

        loss_cls_pos = ce[pos_mask].sum() / num_pos
        loss_cls_neg = ce[~pos_mask].sum() / num_neg
        loss_cls = loss_cls_pos + 0.7 * loss_cls_neg

        if pos_mask.sum() > 0:
            pred_boxes = self._decode_offsets(pred_offsets[pos_mask], anchors[pos_mask])
            target_boxes = self._decode_offsets(target_offsets[pos_mask], anchors[pos_mask])
            loss_bbox = self.bbox_loss(pred_boxes, target_boxes)
        else:
            loss_bbox = torch.tensor(0.0, device=pred_offsets.device)

        return self.weight_bbox * loss_bbox + self.weight_cls * loss_cls

    @staticmethod
    def _decode_offsets(offsets, anchors):
        anchor_w = anchors[:, 2] - anchors[:, 0]
        anchor_h = anchors[:, 3] - anchors[:, 1]
        anchor_cx = (anchors[:, 0] + anchors[:, 2]) / 2
        anchor_cy = (anchors[:, 1] + anchors[:, 3]) / 2

        tx, ty, tw, th = offsets[:, 0], offsets[:, 1], offsets[:, 2], offsets[:, 3]
        cx = anchor_cx + tx * anchor_w
        cy = anchor_cy + ty * anchor_h
        w = torch.exp(tw) * anchor_w
        h = torch.exp(th) * anchor_h

        x1 = cx - w / 2
        y1 = cy - h / 2
        x2 = cx + w / 2
        y2 = cy + h / 2
        return torch.stack([x1, y1, x2, y2], dim=1)
```

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
lr = 1e-3
```

```
model = Detector(num_classes=4, unfreeze_last=1, anchor_sizes=((32,), (64,), (128,), (256,)).to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=20, eta_min=5e-5)
```

```
compute_loss = ComputeLoss(bbox_loss=diou_loss, cls_loss=nn.CrossEntropyLoss(), weight_bbox=5, weight_cls=2)
```

```
runner = Runner(
    model, compute_loss, optimizer, train_dataloader,
    assign_target_tal,
    device=device,
    scheduler=scheduler,
    assign_target_kwargs={"alpha": 6.0, "beta": 1.0, "top_k": 9},
    val_dataloader=test_dataloader,
    score_threshold=0.5,
    nms_threshold=0.5
)
```

```
runner.train(num_epochs=25)
final_loss, final_map = runner.validate()
print(f"Final loss: {final_loss:.4f}, Final mAP: {final_map:.4f}")
```

```
Train epoch: 0%|          | 0/25 [00:00<?, ?it/s]
Process train epoch: 0%|          | 0/58 [00:00<?, ?it/s]
Process train epoch: 2%|          | 1/58 [00:03<03:02, 3.20s/it]
Process train epoch: 3%|          | 2/58 [00:06<03:00, 3.22s/it]
Process train epoch: 5%|          | 3/58 [00:09<02:57, 3.22s/it]
Process train epoch: 7%|          | 4/58 [00:12<02:55, 3.25s/it]
Process train epoch: 9%|          | 5/58 [00:16<02:53, 3.27s/it]
Process train epoch: 10%|         | 6/58 [00:19<02:50, 3.29s/it]
Process train epoch: 12%|         | 7/58 [00:22<02:48, 3.31s/it]
Process train epoch: 14%|         | 8/58 [00:26<02:47, 3.35s/it]
Process train epoch: 16%|         | 9/58 [00:29<02:44, 3.36s/it]
Process train epoch: 17%|         | 10/58 [00:33<02:43, 3.40s/it]
Process train epoch: 19%|         | 11/58 [00:36<02:40, 3.42s/it]
Process train epoch: 21%|         | 12/58 [00:40<02:37, 3.43s/it]
Process train epoch: 22%|         | 13/58 [00:43<02:33, 3.42s/it]
Process train epoch: 24%|         | 14/58 [00:46<02:29, 3.39s/it]
Process train epoch: 26%|         | 15/58 [00:50<02:25, 3.38s/it]
Process train epoch: 28%|         | 16/58 [00:53<02:20, 3.35s/it]
Process train epoch: 29%|         | 17/58 [00:56<02:17, 3.34s/it]
Process train epoch: 31%|         | 18/58 [01:00<02:13, 3.34s/it]
Process train epoch: 33%|         | 19/58 [01:03<02:08, 3.30s/it]
Process train epoch: 34%|         | 20/58 [01:06<02:04, 3.28s/it]
Process train epoch: 36%|         | 21/58 [01:09<02:00, 3.25s/it]
Process train epoch: 38%|         | 22/58 [01:12<01:56, 3.22s/it]
Process train epoch: 40%|         | 23/58 [01:16<01:52, 3.21s/it]
Process train epoch: 41%|         | 24/58 [01:19<01:48, 3.19s/it]
Process train epoch: 43%|         | 25/58 [01:22<01:45, 3.20s/it]
Process train epoch: 45%|         | 26/58 [01:25<01:42, 3.20s/it]
Process train epoch: 47%|         | 27/58 [01:28<01:39, 3.21s/it]
Process train epoch: 48%|         | 28/58 [01:32<01:36, 3.20s/it]
Process train epoch: 50%|         | 29/58 [01:35<01:32, 3.20s/it]
Process train epoch: 52%|         | 30/58 [01:38<01:29, 3.21s/it]
Process train epoch: 53%|         | 31/58 [01:41<01:26, 3.21s/it]
Process train epoch: 55%|         | 32/58 [01:45<01:24, 3.24s/it]
Process train epoch: 57%|         | 33/58 [01:48<01:21, 3.24s/it]
Process train epoch: 59%|         | 34/58 [01:51<01:18, 3.28s/it]
Process train epoch: 60%|         | 35/58 [01:54<01:15, 3.28s/it]
Process train epoch: 62%|         | 36/58 [01:58<01:12, 3.27s/it]
```

```
Process train epoch: 64%|██████████| 37/58 [02:01<01:08, 3.28s/it]
Process train epoch: 66%|██████████| 38/58 [02:04<01:05, 3.29s/it]
Process train epoch: 67%|██████████| 39/58 [02:08<01:02, 3.28s/it]
Process train epoch: 69%|██████████| 40/58 [02:11<00:59, 3.29s/it]
Process train epoch: 71%|██████████| 41/58 [02:14<00:55, 3.29s/it]
Process train epoch: 72%|██████████| 42/58 [02:17<00:52, 3.28s/it]
Process train epoch: 74%|██████████| 43/58 [02:21<00:48, 3.26s/it]
Process train epoch: 76%|██████████| 44/58 [02:24<00:45, 3.25s/it]
Process train epoch: 78%|██████████| 45/58 [02:27<00:42, 3.25s/it]
Process train epoch: 79%|██████████| 46/58 [02:30<00:38, 3.25s/it]
Process train epoch: 81%|██████████| 47/58 [02:34<00:35, 3.23s/it]
Process train epoch: 83%|██████████| 48/58 [02:37<00:32, 3.24s/it]
Process train epoch: 84%|██████████| 49/58 [02:40<00:29, 3.23s/it]
Process train epoch: 86%|██████████| 50/58 [02:43<00:25, 3.24s/it]
Process train epoch: 88%|██████████| 51/58 [02:47<00:22, 3.24s/it]
Process train epoch: 90%|██████████| 52/58 [02:50<00:19, 3.23s/it]
Process train epoch: 91%|██████████| 53/58 [02:53<00:16, 3.22s/it]
Process train epoch: 93%|██████████| 54/58 [02:56<00:12, 3.25s/it]
Process train epoch: 95%|██████████| 55/58 [02:59<00:09, 3.23s/it]
```

Нижe определена вспомогательная функция для валидации качества. Можете использовать `Runner.validate`. Важное уточнение, ей нужен метод для фильтрации предсказаний. Можете тоже скопировать его из семинара, если он у вас не менялся.

```
final_loss, final_map = runner.validate()
print(f"Final loss: {final_loss:.4f}, Final mAP: {final_map:.4f}")

Final loss: 1.1590, Final mAP: 0.0633

model.eval()
images, targets = next(iter(test_dataloader))
images = images.to(device)
with torch.no_grad():
    outputs = model(images)
    preds = runner._filter_predictions(outputs, 0.005, 0.5)
    for i in range(min(3, len(images))):
        print(f"Image {i}: {preds[i]['boxes'].shape[0]} boxes, scores: {preds[i]['scores'][:5]}") # я уже не стал фиксить ничего, много боксов с уверенностью 1

Image 0: 558 boxes, scores: tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
Image 1: 705 boxes, scores: tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
Image 2: 366 boxes, scores: tensor([0.9996, 0.9996, 0.9991, 0.9974, 0.9974])
```

Также, напишите развернутые ответы на следующие вопросы:

Questions:

Какой метод label assignment'a помогает лучше обучаться модели? Почему?

- TAL потому что позволяет выделять хорошие якоря если они одновременно уверенно классифицирует объект и точно определяют его положение
- Какое из сделанных вами улучшений внесло наибольший вклад в качество модели? Как вы думаете, почему это произошло?
 - увеличение разрешения картинки, подбор гиперпараметров (я подумал, что увеличение разрешения может помочь модели различать мелкие объекты, вероятно так и оказалось)
 - (я реализовывал шею, но она не улучшила результат, только ускорила переобучение(может быть я неправильно реализовал, но мне это дз уже надоело), поэтому я удалил)

Какое из сделанных вами улучшений вообще не изменило метрику? Как вы думаете, почему это произошло?

- многие из аугментаций оказались излишне, некоторые только ухудшали результат, вероятнее всего из-за избыточности или несвойственным для датасета изменениям (в показанной выше версии я удалил многие аугментации, модель сильное переобучилась к 25 эпохе, потери вычислялись раз в 5 эпох)
- за шею утверждать не берусь

к 20 эпохе модель показала результат 0.0881, возможно в интервале от 15-25 модель показала результат выше, но я забыл написать сейв весов, а переобучать ещё час я не стал (потери и mAP выводятся раз в 5 эпох для ускорения вычислений)